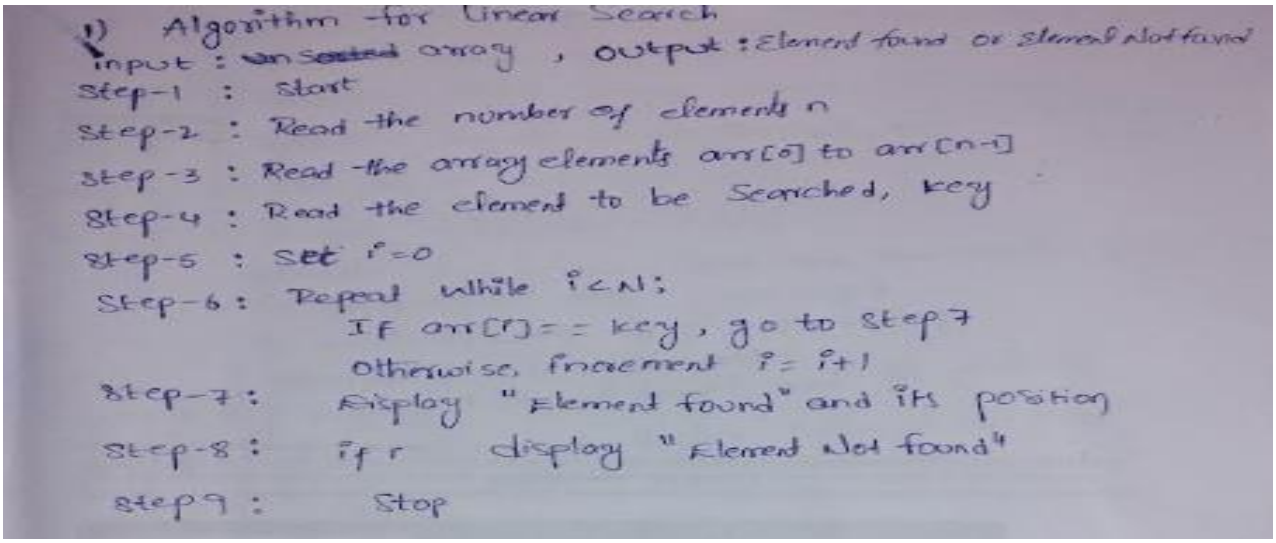
Practical: 2	Implementation and Time analysis of linear and binary search algorithm

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

LINEAR SEARCH



PROGRAM

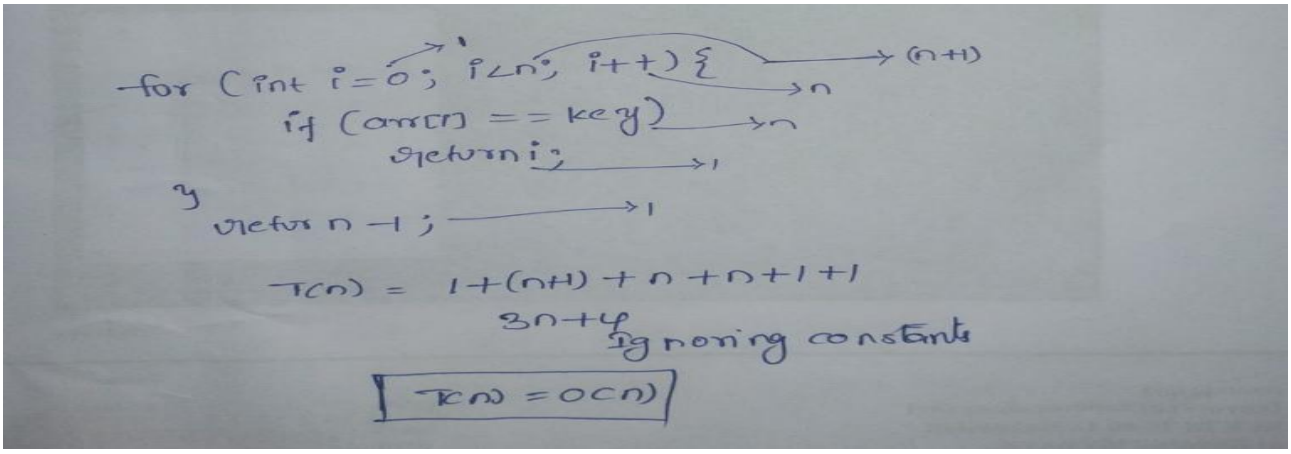
```
#include <iostream>
using namespace std;

int main()
{
   int n, target;

   cout << "Enter number of elements: ";
   cin >> n;

   int arr[n];

   cout << "Enter elements: ";
   for (int i = 0; i < n; i++)
   {
     cin >> arr[i];
   }
   cout << "Enter element to search: ";
```

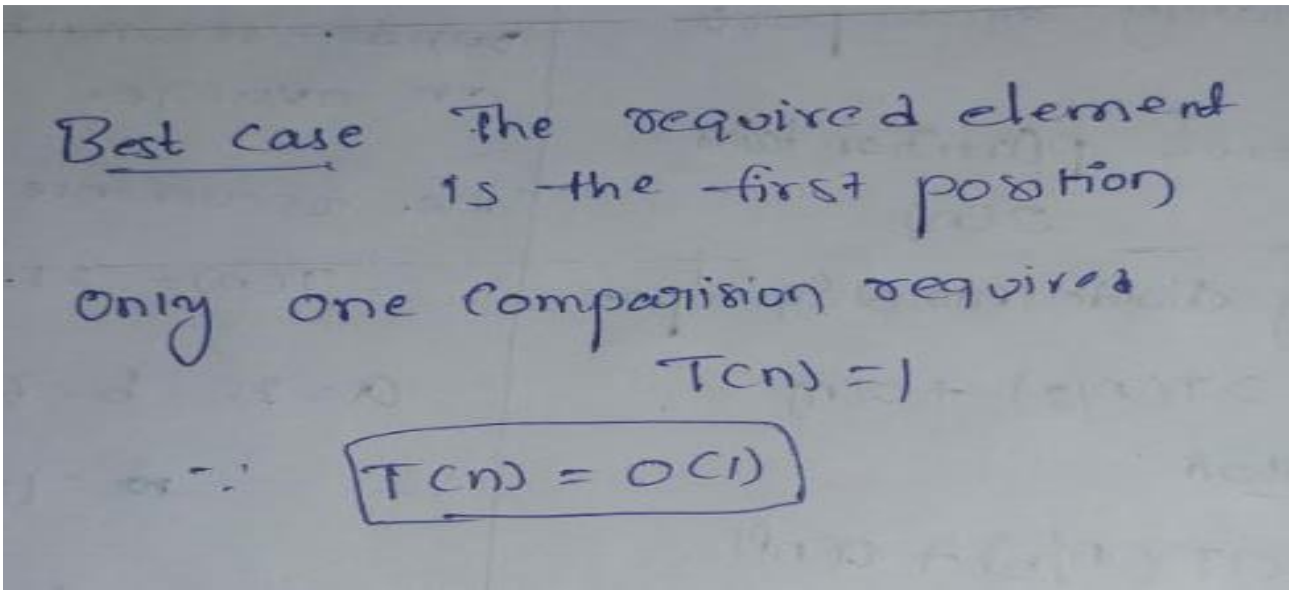
```
    cin >> target;
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == target)
        {
            cout << "Element found at index " << i << endl;
            return 0;
        }
    }
    cout << "Element not found" << endl;

    return 0;
}
```

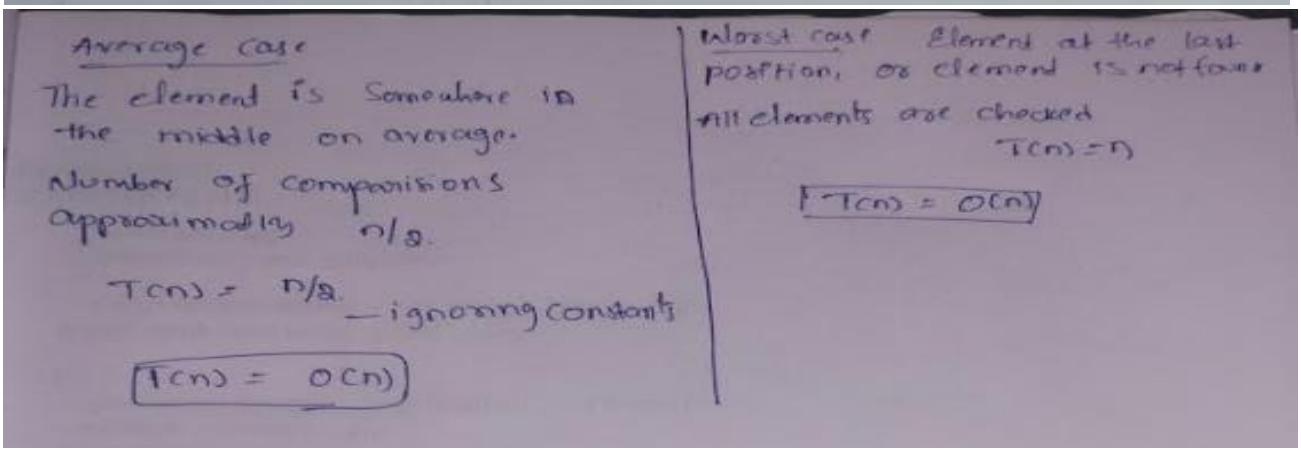
OUTPUT

```
Enter number of elements: 4
Enter elements: 1
2
4
3
Enter element to search: 2
Element found at index 1


=== Code Execution Successful ===
```

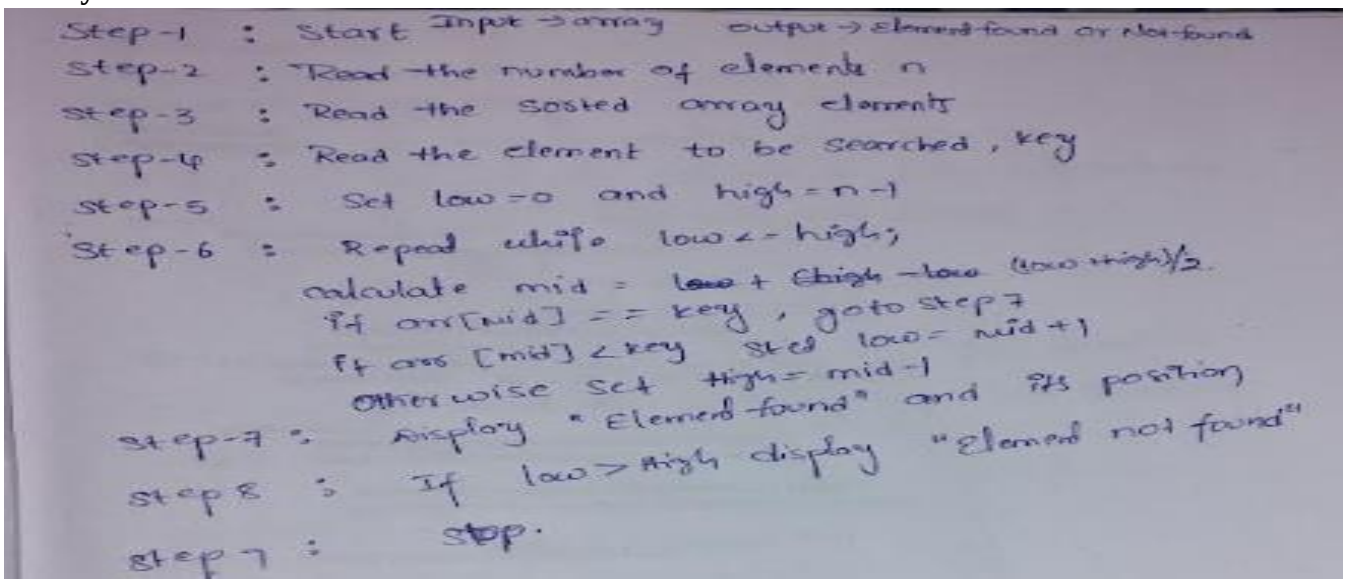
TIME COMPLEXITY




Best case The required element is the first position
Only one comparision required
T(n) = 1
∴ T(n) = O(1)


Average case
The element is somewhere in the middle on average.
Number of comparisons approximately n/2.
T(n) = n/2 — ignoring constants
T(n) = O(n)
Worst case Element at the last position, or element is not found
All elements are checked
T(n) = n
T(n) = O(n)

Binary Search



Program:

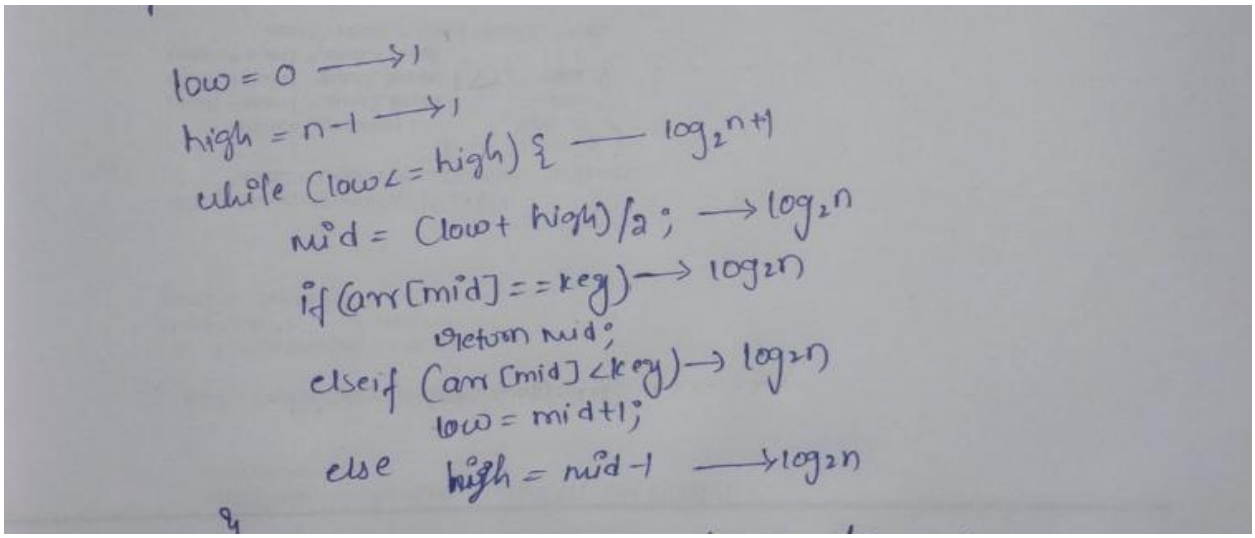
```
#include <iostream>
using namespace std;

int main()
{
   int n, target;

   cout << "Enter number of elements: ";
   cin >> n;

   int arr[n];

   cout << "Enter sorted elements: ";
   for (int i = 0; i < n; i++)
   {
      cin >> arr[i];
   }

   cout << "Enter element to search: ";
   cin >> target;
   int left = 0;
   int right = n - 1;
   while (left <= right)
   {
```

```
        int mid = left + (right - left) / 2;
        if (arr[mid] == target)
        {
            cout << "Element found at index " << mid << endl;
            return 0;
        }
        else if (arr[mid] < target)
        {
            left = mid + 1;
        }
        else
        {
            right = mid - 1;
        }
    }
    cout << "Element not found" << endl;
    return 0;
}
```

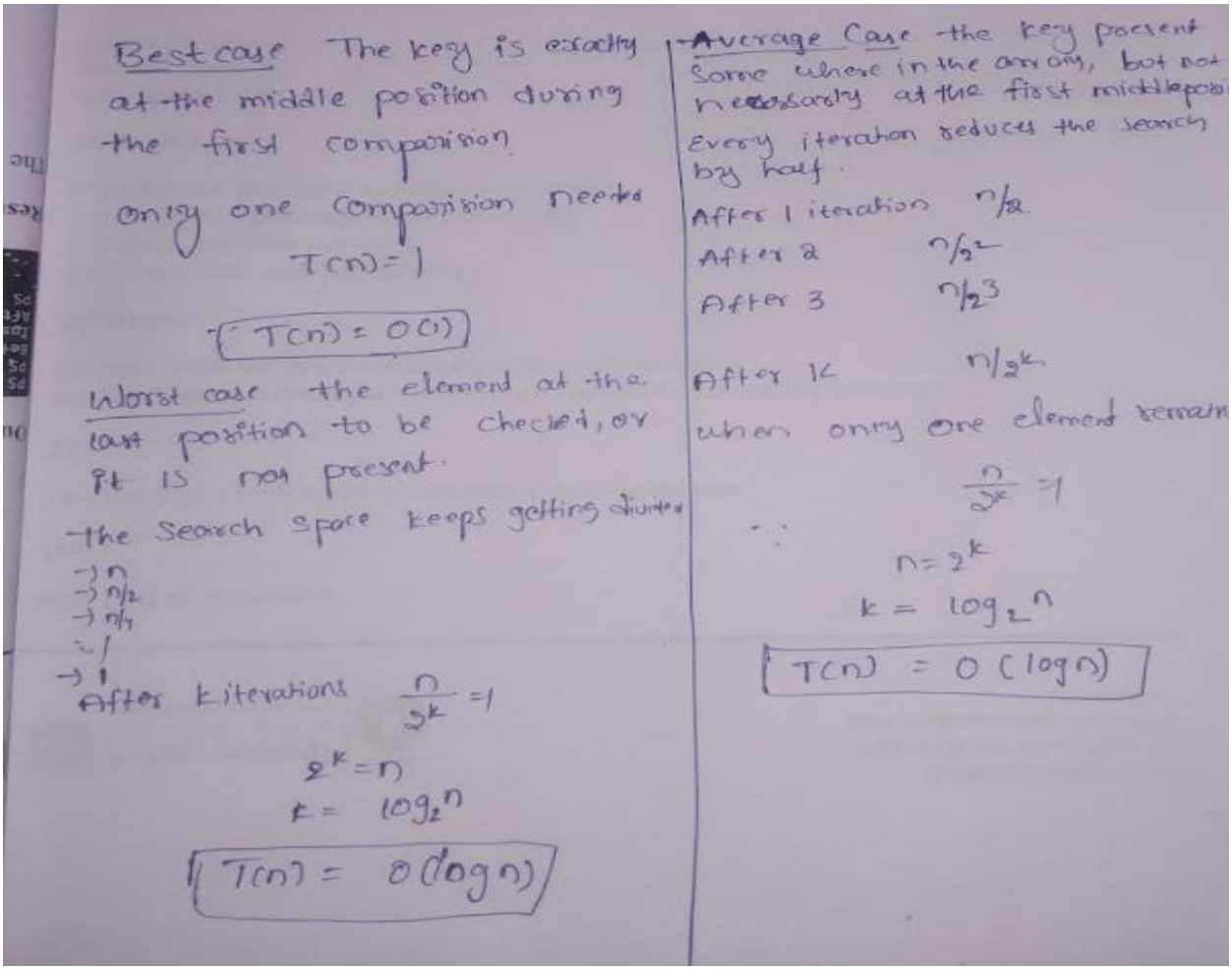
OUTPUT

```
Enter number of elements: 4
Enter sorted elements: 11
22
33
44
Enter element to search: 33
Element found at index 2


=== Code Execution Successful ===
```

TIME COMPLEXITY



```
low = 0 ------> 1
high = n-1 ------> 1
while (low <= high) {  ------ log2n+1
    mid = (low + high)/2;  ------> log2n
    if (arr[mid] == key) ------> log2n
        return mid;
    elseif (arr[mid] < key) ------> log2n
        low = mid+1;
    else  high = mid-1  ------> log2n
}
```



Best case The key is exactly at the middle position during the first comparision

Only one comparision needed

$T(n) = 1$

$T(n) = O(1)$

Worst case the element at the last position to be checked, or it is not present.

-the search space keeps getting divided

→ n
→ n/2
→ n/4
:
→ 1

After k iterations $\frac{n}{2^k} = 1$

$2^k = n$

$k = \log_2 n$

$T(n) = O(\log n)$

Average Case the key present some where in the array, but not necessarily at the first middle position

Every iteration reduces the search by half.

After 1 iteration $n/2$

After 2 $n/2^2$

After 3 $n/2^3$

After k $n/2^k$

when only one element remains

$\frac{n}{2^k} = 1$

$\therefore$

$n = 2^k$

$k = \log_2 n$

$T(n) = O(\log n)$

Conclusion:

From this implementation, I understood that the searching method depends on the type of data. Linear Search is simple and checks elements one by one, while Binary Search is faster because it repeatedly divides the sorted array into halves.